

Laboratory Guide 3.6: Shor's Algorithm

Quantum Factoring via Period Finding and the QFT

Course: Quantum Computing Foundation

Contents

1	Introduction and Objectives	2
2	Part A: The Classical Foundation	2
2.1	RSA and the Factoring Problem	2
2.2	Reduction: Factoring $\Rightarrow$ Order Finding	2
3	Part B: The Quantum Fourier Transform	3
3.1	Definition	3
3.2	Circuit Structure	4
3.3	The QFT and Periodicity	4
4	Part C: The Quantum Period-Finding Subroutine	5
4.1	Circuit Overview	5
4.2	Full State-Vector Evolution ($N = 15, a = 7, n = 4$)	5
4.3	Continued-Fractions Extraction	6
5	Part D: Building the QFT (IBM Quantum Composer)	6
5.1	Exercise 1: Build and Verify a 2-Qubit QFT	6
5.2	Composer Activity: Period Detection (Conceptual)	7
6	Part E: Coding Shor's Algorithm (Qiskit)	7
6.1	Exercise 2: Python Implementation	7
7	Part F: Assessment (Quiz)	10

1 Introduction and Objectives

In Lab 3 and Lab 3.5 we learned to exploit quantum interference for global function evaluation (Deutsch’s algorithm, with its Deutsch–Jozsa extension) and unstructured search (Grover). In this laboratory we tackle the problem that gave quantum computing much of its public profile: **integer factorisation**. Shor’s Algorithm (1994) factors an n -bit integer N in time $\mathcal{O}(n^3)$, while the best known classical algorithms require $e^{\mathcal{O}(n^{1/3}(\log n)^{2/3})}$ (the General Number Field Sieve), which grows faster than any power of n . This separation underpins the security of every RSA-encrypted message on the internet today.

Prerequisites:

- Lab 3.5: *Grover’s Algorithm and Amplitude Amplification*
- Lesson QC14: *Quantum Algorithms (Grover & Shor)*
- Basic modular arithmetic (congruences, gcd)

Learning Objectives:

1. Understand the classical reduction from factoring to *order finding*.
2. Master the Quantum Fourier Transform (QFT) — its matrix, circuit, and role.
3. Trace the full state-vector evolution of the period-finding subroutine.
4. Implement a complete Shor simulation in Qiskit for $N = 15$, $a = 7$.
5. Interpret measured bitstrings via the continued-fractions algorithm.

Important: The Exponential Speedup

The Exponential Speedup — What Is Really at Stake

RSA-2048 (still widely used to secure HTTPS connections) relies on the fact that factoring a 2048-bit number is computationally infeasible classically. The GNFS would take roughly 10^{30} – 10^{35} operations (estimates vary): even at 10^{18} operations per second, an exascale supercomputer, that is between tens of thousands and billions of years. Shor’s Algorithm would factor the same number in $\sim (2048)^3 \approx 8.6 \times 10^9$ quantum operations — a few hours on a fault-tolerant quantum computer. The classical cost grows faster than any power of the number of bits n ; Shor’s grows only like n^3 .

2 Part A: The Classical Foundation

2.1 RSA and the Factoring Problem

RSA encryption is built on a simple asymmetry: **multiplying** two large primes p and q to produce $N = p \cdot q$ is trivially fast, but **factoring** N back into p and q is (classically) very hard: the best known method needs a time that grows faster than any power of the number of bits. The security of the system relies entirely on this asymmetry.

2.2 Reduction: Factoring $\Rightarrow$ Order Finding

The key mathematical insight behind Shor’s Algorithm is that the hard problem of factoring can be *reduced* to the (classically hard but *quantum-easy*) problem of finding the **multiplicative order** of an integer.

Multiplicative Order

Let $N \in \mathbb{Z}^+$ and $a \in \mathbb{Z}$ with $\gcd(a, N) = 1$. The **multiplicative order** (or **period**) of a modulo N is the smallest positive integer r such that

$$a^r \equiv 1 \pmod{N}.$$

Equivalently, r is the period of the function $f(x) = a^x \bmod N$.

The full classical reduction proceeds in three steps:

1. **Choose** a random a with $2 \leq a \leq N - 1$. Check $g = \gcd(a, N)$. If $g > 1$, we already found a factor — done.
2. **Find the order** r of a modulo N (this is the quantum step).
3. **Extract the factors:** If r is even and $a^{r/2} \not\equiv -1 \pmod{N}$, then N divides $a^r - 1 = (a^{r/2} - 1)(a^{r/2} + 1)$ but divides neither factor (r is the smallest exponent with $a^r \equiv 1$, and $a^{r/2} \not\equiv -1$), so

$$p = \gcd(a^{r/2} - 1, N), \quad q = \gcd(a^{r/2} + 1, N)$$

are non-trivial factors of N .

Classical Reduction: $N = 15$, $a = 7$

1. $\gcd(7, 15) = 1$ — no trivial factor; proceed to step 2.
2. Compute $f(x) = 7^x \bmod 15$:

x	0	1	2	3	4	5	6	7	8
$f(x)$	1	7	4	13	1	7	4	13	1

The sequence repeats with period $r = 4$.

3. $r = 4$ is even; check $7^{4/2} = 7^2 = 49 \equiv 4 \not\equiv -1 \pmod{15}$. Condition satisfied.

$$p = \gcd(7^2 - 1, 15) = \gcd(48, 15) = 3$$

$$q = \gcd(7^2 + 1, 15) = \gcd(50, 15) = 5$$

Verification: $3 \times 5 = 15$ ✓

3 Part B: The Quantum Fourier Transform

3.1 Definition

The **Quantum Fourier Transform** (QFT) is the quantum analogue of the Discrete Fourier Transform (DFT). Acting on an n -qubit basis state $|x\rangle$:

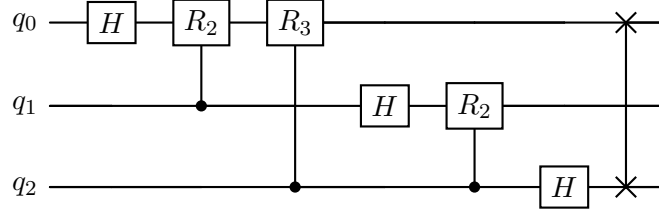
$$\text{QFT}_N |x\rangle = \frac{1}{\sqrt{N}} \sum_{k=0}^{N-1} \omega^{xk} |k\rangle, \quad \omega = e^{2\pi i/N}, \quad N = 2^n. \quad (1)$$

It maps each basis state to a uniform superposition with phases that encode the frequency content of x . Critically, it is *unitary* and can be implemented with only $\mathcal{O}(n^2)$ gates (vs. $\mathcal{O}(N \log N)$ for the classical FFT).

3.2 Circuit Structure

The QFT on n qubits decomposes into a cascade of Hadamard gates and controlled phase-rotation gates R_k , where $R_k = \begin{pmatrix} 1 & 0 \\ 0 & e^{2\pi i/2^k} \end{pmatrix}$, followed by a bit-reversal via SWAP gates.

The circuit for $n = 3$ qubits is shown below:



The figure uses the textbook ordering (top wire = most significant bit). The code in Part E uses Qiskit's order (qubit 0 = least significant bit), so there the same circuit appears mirrored, followed by the same final swap. The IBM Composer also labels its bit strings in Qiskit's order (q_0 = rightmost bit): in the Composer exercises of Part D, the state written here as $|01\rangle$ ($q_0 = 0$, $q_1 = 1$) is displayed as 10.

3.3 The QFT and Periodicity

The key property we exploit is: *the QFT maps a periodic state to concentrated peaks in frequency space.*

Periodic State

A state of the form $|\psi_r\rangle = \frac{1}{\sqrt{K}} \sum_{j=0}^{K-1} |jr + s\rangle$ (with period r , offset s , and $K = \lfloor N/r \rfloor$) has amplitudes concentrated at multiples of N/r after applying QFT_N .

QFT of a Periodic State ($r = 4$, $N = 16$)

Take $|\psi_0\rangle = \frac{1}{2}(|0\rangle + |4\rangle + |8\rangle + |12\rangle)$. Applying QFT_{16} :

$$\text{QFT}_{16} |\psi_0\rangle = \frac{1}{2} \cdot \frac{1}{4} \sum_{k=0}^{15} \left[\sum_{j=0}^3 e^{2\pi i(4j)k/16} \right] |k\rangle = \frac{1}{8} \sum_{k=0}^{15} \left[\sum_{j=0}^3 e^{\pi i j k / 2} \right] |k\rangle.$$

The inner sum is a geometric series:

$$\sum_{j=0}^3 e^{\pi i j k / 2} = \begin{cases} 4 & \text{if } k \equiv 0 \pmod{4} \\ 0 & \text{otherwise.} \end{cases}$$

Therefore, $\text{QFT}_{16} |\psi_0\rangle = \frac{1}{2}(|0\rangle + |4\rangle + |8\rangle + |12\rangle)$, with measurement probabilities concentrated at $\{0, 4, 8, 12\} = \{k \cdot N/r \mid k = 0, 1, 2, 3\}$. **The period $r = 4$ is encoded in the spacing of the peaks.**

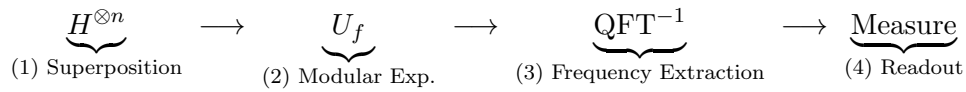
4 Part C: The Quantum Period-Finding Subroutine

4.1 Circuit Overview

The algorithm uses two quantum registers:

- **Counting register** (n qubits): stores the superposition of exponents $x \in \{0, \dots, 2^n - 1\}$. In general we need $2^n \geq N^2$ (for $N = 15$: $n = 8$). This lab uses $n = 4$ ($2^4 = 16$), a declared simplification: it works for $N = 15$ because every period here (2 or 4) divides 16 exactly.
- **Work register** (m qubits, $m = \lceil \log_2 N \rceil$): stores $f(x) = a^x \bmod N$. For $N = 15$, $m = 4$ qubits.

The circuit has three stages:



4.2 Full State-Vector Evolution ($N = 15$, $a = 7$, $n = 4$)

Complete State Evolution

Step 1 — Superposition. Apply $H^{\otimes 4}$ to the counting register, work register initialised to $|1\rangle$:

$$|\psi_1\rangle = \frac{1}{4} \sum_{x=0}^{15} |x\rangle_C \otimes |1\rangle_W.$$

Step 2 — Oracle (Modular Exponentiation). The unitary $U_f |x\rangle_C |y\rangle_W = |x\rangle_C |y \cdot a^x \bmod N\rangle_W$ entangles the registers:

$$|\psi_2\rangle = \frac{1}{4} \sum_{x=0}^{15} |x\rangle_C \otimes \underbrace{|7^x \bmod 15\rangle_W}_{\text{cycles: } 1, 7, 4, 13, 1, 7, \dots}.$$

The work register cycles through $\{1, 7, 4, 13\}$ with period $r = 4$.

Step 3 — Measure work register (conceptual). Suppose the outcome is $|1\rangle_W$ (this occurs for $x \in \{0, 4, 8, 12\}$). The counting register collapses to the uniform periodic superposition:

$$|\psi_3\rangle = \frac{1}{2} (|0\rangle + |4\rangle + |8\rangle + |12\rangle)_C \otimes |1\rangle_W.$$

Note: in the actual circuit the work register is never measured: measuring only the counting register at the end gives the same outcome statistics, so this measurement only simplifies the algebra; the circuit does not need it.

Step 4 — Inverse QFT. As computed in Part B, the IQFT concentrates probability at $\{0, 4, 8, 12\} = \{k \cdot 16/4 \mid k = 0, 1, 2, 3\}$:

$$\text{QFT}_{16}^{-1} |\psi_3\rangle = \frac{1}{2} (|0\rangle + |4\rangle + |8\rangle + |12\rangle)_C.$$

Before the QFT, the counting register *marginal* was uniform ($1/16$ for each state). The QFT concentrates the probability into $r = 4$ peaks — this is the quantum advantage.

Step 5 — Measurement and Classical Post-processing. One of $\{0, 4, 8, 12\}$ is measured with equal probability $1/4$. The measured value y encodes the phase $\phi = y/2^n$.

4.3 Continued-Fractions Extraction

Continued-Fractions Algorithm

Given measured value y , compute the best rational approximation $p/q \approx y/2^n$ with denominator $q \leq N$. The denominator q is the period estimate $\hat{r}$.

Example outcomes:

y	$y/16$	Best approx	$\hat{r}$	$7^{\hat{r}} \bmod 15$	Result
0	0	0/1	1	7	Retry (trivial)
4	1/4	1/4	4	1	✓
8	1/2	1/2	2	4	Try $2\hat{r} = 4$: ✓
12	3/4	3/4	4	1	✓

Success probability per run: $\sim 75\%$. Expected trials to success: ~ 1.3 .

5 Part D: Building the QFT (IBM Quantum Composer)

Platform: IBM Quantum Composer — <https://quantum.ibm.com/composer>

5.1 Exercise 1: Build and Verify a 2-Qubit QFT

The 2-qubit QFT maps: $|00\rangle \mapsto \frac{1}{2}(|00\rangle + |01\rangle + |10\rangle + |11\rangle)$, $|01\rangle \mapsto \frac{1}{2}(|00\rangle + i|01\rangle - |10\rangle - i|11\rangle)$, etc.

Student Activity

Circuit Instructions:

1. Start with both qubits in $|0\rangle$. A new Composer circuit has four qubits: in the OpenQASM editor change `qubit[4] q;` to `qubit[2] q;` and `bit[4] c;` to `bit[2] c;` (or use *Edit* $\rightarrow$ *Manage registers*). With four qubits every label has four digits (0000, 0001, ...).
2. Apply H to q_0 (MSB).
3. Apply S ($= R_2 = P(\pi/2)$) to q_0 , controlled on q_1 .
4. Apply H to q_1 .
5. Apply SWAP between q_0 and q_1 (bit-reversal).
6. Do **not** add a measurement, and look at the Q-Sphere. Once a circuit contains a measurement, the Composer's live views show a single simulated run (one dot, one bar at 100%). Tick *Phase angle* under the Q-Sphere to read the phases.

Expected: All four states $|00\rangle, |01\rangle, |10\rangle, |11\rangle$ with equal probability 25%. Now **change the initial state to $|01\rangle$** (add an X gate on q_1 before step 2) and verify the phases rotate as predicted.

5.2 Composer Activity: Period Detection (Conceptual)

Student Activity

Observe the Oracle Effect:

1. Set the register back to four qubits (qubit[4] q; and bit[4] c;, the default of a new circuit) and put them in uniform superposition (H on all).
2. Inspect the Q-Sphere — all 16 states equally bright.
3. Now apply a phase oracle: apply Z to q_0 (MSB) only. This flips the sign of every state with $q_0 = 1$ (half of the 16 states), creating a phase structure.
4. Apply the H gate to q_0 again and observe that $q_0 = 1$ now has probability 100% while $q_0 = 0$ vanishes ($HZH = X$): the sign pattern has become a definite outcome — a 1-qubit Fourier transform! (Without the Z , $HH = I$ gives $q_0 = 0$ with certainty.)

Insight: The full Shor oracle followed by QFT does exactly this across all n qubits simultaneously, but for the *period- r subspace*.

6 Part E: Coding Shor's Algorithm (Qiskit)

6.1 Exercise 2: Python Implementation

The full implementation searches for the period of $f(x) = 7^x \bmod 15$. The circuit uses 4 counting qubits and a 4-qubit work register that starts at the number 1. Counting qubit j multiplies the work register by $7^{2^j} \bmod 15$: by 7 for $j = 0$ and by 4 for $j = 1$. Qubits $j = 2, 3$ need no gate at all, because $7^4 \equiv 7^8 \equiv 1 \pmod{15}$. Each multiplication modulo 15 is built from swaps and bit flips: doubling a 4-bit number modulo 15 simply moves every bit one place up, with the top bit wrapping around to the bottom.

The powers $7^{2^j} \bmod 15$ are computed classically beforehand (by repeated squaring), as in any real run of Shor's algorithm. For a realistic N it is extremely unlikely that any of them turns out to be 1; here the tiny $N = 15$ lets us see $7^4 \equiv 1$ directly — a reminder that this is a toy instance, not a way of knowing r in advance.

Copy the code from the notebook (Open in Colab) or from the lab page, not from this PDF: long lines are wrapped here and quotes are typeset, so text copied from the PDF will not run.

Coding Task: QFT Circuit

```
1 import math, fractions
2 import numpy as np
3 from qiskit import QuantumCircuit, QuantumRegister, ClassicalRegister
4 from qiskit_aer import AerSimulator
5 from qiskit.quantum_info import Statevector, Operator
6
7 def build_qft(n):
8     """Quantum Fourier Transform on n qubits.
9     Qubit 0 is the least significant bit (Qiskit's order), so the integer
10     x
11     is read from a measured bitstring with int(bitstring, 2).
12     Maps |x> to (1/sqrt(2^n)) * sum over k of exp(2*pi*i * x*k / 2^n) |k>.
13     """
14     qc = QuantumCircuit(n, name='QFT')
```

```

13     for j in reversed(range(n)):          # start from the most
14         significant qubit
15         qc.h(j)
16         for k in reversed(range(j)):      # controlled phase rotations
17             from the lower qubits
18             qc.cp(math.pi / 2 ** (j - k), k, j)
19         for i in range(n // 2):          # swaps put the output bits in
20             the right order
21             qc.swap(i, n - 1 - i)
22         return qc
23
24 # 1) Verify: the circuit's matrix equals the Fourier-transform matrix
25 n_qubits = 3
26 M = 2 ** n_qubits
27 fourier = np.array([[np.exp(2j * np.pi * x * k / M) for x in range(M)]
28                     for k in range(M)]) / np.sqrt(M)
29 print("QFT circuit equals the Fourier matrix:",
30       np.allclose(Operator(build_qft(n_qubits)).data, fourier))
31
32 # 2) Apply the QFT to |001> (decimal 1) and look at the phases
33 qc_qft = QuantumCircuit(n_qubits)
34 qc_qft.x(0)                                # qubit 0 = least significant
35 bit -> |001>
36 qc_qft.compose(build_qft(n_qubits), inplace=True)
37 state = Statevector.from_instruction(qc_qft)
38
39 print("QFT|001>: every outcome has probability 1/8; the phase grows by 1/8
40       of a turn per step:")
41 for k, amp in enumerate(state.data):
42     turns = (np.angle(amp) / (2 * np.pi)) % 1
43     print(f" |{k:03b}> (k={k}) probability = {abs(amp)**2:.3f} phase
44           = {round(turns*8)}/8 of a turn")

```

Coding Task: Full Order-Finding Circuit for $N = 15$, $a = 7$

```

1 N, a = 15, 7
2 n_count = 4          # counting register: 4 qubits -> measured values y = 0
3 ... 15
4 n_work = 4           # work register: 4 qubits hold one number 0 ... 15
5
6 def c_mult_mod15(qc, ctrl, w, factor):
7     """If qubit 'ctrl' is 1, multiply the number held in the work qubits w
8     by 'factor', modulo 15.
9     w[0] is the least significant bit. Works for the values 1 ... 14, the
10    only ones that occur here.
11    Trick: doubling mod 15 moves every bit one place up, the top bit
12    wrapping to the bottom, so
13    x4 = move every bit two places (two swaps);
14    x7 = x(-8) = move every bit three places, then flip all bits (15 - z
15    = NOT z for 4 bits)."""
16    if factor == 1:
17        return                                # multiplying by 1 changes
18        nothing
19    if factor == 4:
20        qc.cswap(ctrl, w[0], w[2])
21        qc.cswap(ctrl, w[1], w[3])
22    elif factor == 7:
23        qc.cswap(ctrl, w[0], w[1])
24        qc.cswap(ctrl, w[1], w[2])
25        qc.cswap(ctrl, w[2], w[3])

```

```

20         for q in w:
21             qc.cx(ctrl, q)
22     else:
23         raise ValueError("for a = 7 only the factors 1, 4 and 7 are needed")
24
25 # Check the arithmetic on every allowed input before trusting it
26 for factor in (7, 4):
27     for y in range(1, 15):
28         test = QuantumCircuit(5)                # qubit 0 = control,
29         qubits 1-4 = work
30         test.x(0)
31         for b in range(4):
32             if (y >> b) & 1:
33                 test.x(1 + b)
34         c_mult_mod15(test, 0, [1, 2, 3, 4], factor)
35         out = int(np.argmax(np.abs(Statevector(test).data))) >> 1
36         assert out == (factor * y) % N
37 print("Checked: the gates multiply by 7 and by 4 modulo 15 on every input
38     1 ... 14.")
39
40 def build_shor_circuit():
41     c = QuantumRegister(n_count, 'count')
42     w = QuantumRegister(n_work, 'work')
43     cr = ClassicalRegister(n_count, 'c')
44     qc = QuantumCircuit(c, w, cr)
45
46     qc.h(c)                # counting register: equal superposition of x = 0
47     ... 15
48     qc.x(w[0])            # work register starts at the number 1 (|0001>)
49     # Counting qubit j multiplies the work register by a^(2^j) mod 15.
50     # Together they multiply by a^x, so the work register ends up holding
51     7^x mod 15.
52     # j=0: 7^1 = 7      j=1: 7^2 = 49 = 4 (mod 15)    j=2: 7^4 = 1      j=3:
53     7^8 = 1
54     for j in range(n_count):
55         c_mult_mod15(qc, c[j], w, pow(a, 2 ** j, N))
56     qc.barrier()
57     qc.compose(build_qft(n_count).inverse(), qubits=c, inplace=True) #
58     inverse QFT
59     qc.measure(c, cr)
60     return qc

```

Coding Task: Run and Extract the Period

```

1 qc_shor = build_shor_circuit()
2 exact = Statevector(qc_shor.remove_final_measurements(inplace=False)).
3     probabilities_dict(qargs=range(n_count))
4 print("Exact probabilities:", {str(k): round(float(v), 3) for k, v in
5     sorted(exact.items()) if v > 1e-9})
6
7 counts = AerSimulator(seed_simulator=1234).run(qc_shor, shots=1000).result
8     ().get_counts()
9 print("Shor results (1000 shots, seeded):", dict(sorted(counts.items())))
10
11 print(f"{'bits':>6} {'y':>3} {'y/16':>6} {'fraction':>9}    result")
12 factors = set()
13 for bits in sorted(counts, key=lambda b: int(b, 2)):
14     y = int(bits, 2)                # counting
15     register as an integer

```

```

12     frac = fractions.Fraction(y, 2 ** n_count).limit_denominator(N - 1)
13     r = frac.denominator                                # candidate
14     period
15     if y == 0:
16         result = "phase 0 carries no information -> run again"
17     else:
18         note = ""
19         while pow(a, r, N) != 1 and r < N:                # denominator too
20             small? try its multiples
21             note = f"{a}^{frac.denominator} mod {N} = {pow(a, frac.
22             denominator, N)}, not 1 -> try multiples; "
23             r += frac.denominator
24             if pow(a, r, N) != 1:
25                 result = "no period found from this outcome -> run again"
26             elif r % 2 == 1 or pow(a, r // 2, N) == N - 1:
27                 result = f"r = {r} cannot be used -> choose another a"
28             else:
29                 p, q = math.gcd(a ** (r // 2) - 1, N), math.gcd(a ** (r // 2)
30                 + 1, N)
31                 result = note + f"r = {r}: gcd({a}^{r//2} - 1, {N}) = {p},
32                 gcd({a}^{r//2} + 1, {N}) = {q}"
33                 factors |= {p, q}
34             print(f"{bits:>6} {y:>3} {str(y)+'/16':>6} {str(frac):>9} {result}
35                 [{counts[bits]} shots]")
36
37 print("\nFactors found:", sorted(factors), "->", " x ".join(map(str,
38 sorted(factors))), "=", math.prod(factors))

```

Expected output: Peaks at 0000, 0100, 1000, 1100 (values 0,4,8,12), each $\approx 25\%$.
 Extracted period $r = 4$. Factors: $\{3, 5\}$.

7 Part F: Assessment (Quiz)

Q1. The Speedup

What is the time complexity of Shor's Algorithm for factoring an n -bit integer?

- A) $\mathcal{O}(2^n)$
- B) $\mathcal{O}(n^3)$
- C) $\mathcal{O}(\sqrt{n})$

Q2. Role of the QFT

Why is the inverse QFT applied to the counting register at the end of the algorithm?

- A) To disentangle the counting and work registers before measurement.
- B) To convert the uniform probability distribution of the counting register into peaks at multiples of $2^n/r$, from which the period r can be efficiently extracted.
- C) To apply amplitude amplification as in Grover's Algorithm.

Q3. Failure Mode

Shor's Algorithm is run for $N = 15$, $a = 7$, $n = 4$. A measurement yields $y = 0$. What should the algorithm do?

- A) Report $r = 0$ and halt.

B) Report $r = N = 15$.

C) Discard the result and re-run; $y = 0$ is an uninformative outcome.

Q4. Continued Fractions

A measurement returns $y = 8$ from $n = 4$ counting qubits ($2^n = 16$). What period $\hat{r}$ does the continued-fractions algorithm return, and how should it be interpreted?

A) $\hat{r} = 2$; since $7^2 \not\equiv 1 \pmod{15}$, try $2\hat{r} = 4$.

B) $\hat{r} = 4$; directly compute factors.

C) $\hat{r} = 8$; the measurement is a higher harmonic.

Solutions: Q1: B ($\mathcal{O}(n^3)$, polynomial in the number of bits of N); Q2: B (the QFT converts periodic structure into measurable frequency peaks); Q3: C ($y = 0$ gives $\phi = 0$ whose CF expansion yields $r = 1$, trivially true for all a — no information gained, repeat); Q4: A ($8/16 = 1/2 \Rightarrow \hat{r} = 2$; verify $7^2 \bmod 15 = 4 \neq 1$, so try $r = 2\hat{r} = 4$; confirm $7^4 \bmod 15 = 1$).